

日K 五十音直覺認讀訓練器（重構版）

下面是一份完整可直接覆蓋的 `index.html`。

這版重構重點：

- 修正 Gemini API 呼叫穩定性
- 加入 429 Retry Backoff
- 自動 fallback model
- 改善 JSON 清洗
- 修正五十音 typo
- 加入 3 秒後顯示答案模式
- 加入詞頻權重抽卡
- 更穩定的 localStorage schema
- 增加 debug log
- 改善 UI 狀態顯示
- 更適合未來改 Electron
- 支援 localhost 環境

建議執行方式：

```
python -m http.server 8080
```

然後開：

```
http://localhost:8080
```

不要直接雙擊 file:// 執行。

```
<!DOCTYPE html>
<html lang="zh-TW">
<head>
<meta charset="UTF-8" />
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
<title>日K五十音直覺認讀訓練器</title>

<script src="https://cdn.tailwindcss.com"></script>

<style>
@import url('https://fonts.googleapis.com/css2?
family=Noto+Sans+JP:wght@400;700&display=swap');

body {
  font-family: 'Noto Sans JP', sans-serif;
  background: #0b0f19;
```

```

}

.kana-glow {
  text-shadow:
    0 0 12px rgba(0,229,255,.5),
    0 0 32px rgba(0,229,255,.25);
}

.fade-in {
  animation: fadeIn .25s ease;
}

@keyframes fadeIn {
  from {
    opacity: 0;
    transform: translateY(8px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}

::-webkit-scrollbar {
  width: 8px;
}

::-webkit-scrollbar-thumb {
  background: #1f2937;
  border-radius: 999px;
}
</style>
</head>

<body class="text-gray-100 min-h-screen overflow-hidden">

<div class="flex flex-col lg:flex-row h-screen">

  <!-- 左側 -->
  <aside class="w-full lg:w-80 bg-gray-900 border-r border-gray-800 p-5 overflow-y-auto shrink-0">

    <div class="mb-6">
      <h1 class="text-2xl font-bold text-cyan-400 tracking-widest">
        🎵 日K五十音認讀器
      </h1>
      <p class="text-xs text-gray-500 mt-2 leading-relaxed">
        目標：把日文假名訓練成像注音一樣的直覺反應。
      </p>
    </div>
  </aside>

```

```

<div class="space-y-5">

  <div>
    <label class="text-xs text-gray-400 block mb-2">
      Gemini API Key
    </label>

    <div class="relative">
      <input
        id="apiKeyInput"
        type="password"
        placeholder="AlzaSy..."
        class="w-full bg-gray-950 border border-gray-800 rounded-lg px-3 py-2
pr-10 text-sm text-cyan-300 focus:outline-none focus:border-cyan-500"
      />

      <button
        id="toggleKeyBtn"
        class="absolute right-3 top-2 text-gray-500 hover:text-white"
      >

      </button>
    </div>
  </div>

  <div>
    <label class="text-xs text-gray-400 block mb-2">
      字庫關鍵字
    </label>

    <input
      id="keywordInput"
      type="text"
      placeholder="熱血動漫、JPOP高頻歌詞、YOASOBI..."
      class="w-full bg-gray-950 border border-gray-800 rounded-lg px-3 py-2 text-sm focus:outline-none
focus:border-cyan-500"
    />
  </div>

  <div class="grid grid-cols-2 gap-2">
    <button
      id="fetchBtn"
      class="bg-cyan-600 hover:bg-cyan-500 transition rounded-lg py-3 font-bold"
    >
      AI 擴充字庫
    </button>

    <button
      id="clearBtn"
      class="bg-red-700 hover:bg-red-600 transition rounded-lg py-3 font-bold"

```

```

        >
        清空字庫
    </button>
</div>

<div class="bg-gray-950 border border-gray-800 rounded-xl p-4 text-sm space-y-2">
    <div class="flex justify-between">
        <span class="text-gray-400">字庫數量</span>
        <span id="vocabCount" class="text-cyan-400 font-bold">0</span>
    </div>

    <div class="flex justify-between">
        <span class="text-gray-400">模式</span>
        <span class="text-emerald-400">假名反射訓練</span>
    </div>
</div>

<div class="bg-gray-950 border border-gray-800 rounded-xl p-4">
    <div class="flex items-center justify-between mb-3">
        <span class="text-sm text-gray-300 font-bold">
            3 秒後顯示答案
        </span>

        <input id="delayRevealToggle" type="checkbox" checked
class="scale-125" />
    </div>

    <p class="text-xs text-gray-500 leading-relaxed">
        先看假名，在腦中默念發音，再顯示 Romaji 與中文。
    </p>
</div>

</div>
</aside>

<!-- 中央 -->
<main class="flex-1 flex flex-col justify-between items-center p-6 overflow-y-auto bg-gray-950">

    <div class="w-full max-w-3xl">
        <div
            id="statusBox"
            class="bg-gray-900 border border-gray-800 rounded-xl px-4 py-3 text-sm text-
gray-300"
        >
            系統初始化完成。
        </div>
    </div>

    <div class="flex-1 flex items-center justify-center w-full py-10">

        <div class="w-full max-w-3xl bg-gradient-to-b from-gray-900 to-black border border-

```

```

gray-800 rounded-3xl p-10 shadow-2xl relative overflow-hidden">

    <div class="absolute inset-0 bg-cyan-500/5"></div>

    <div class="relative z-10 text-center fade-in">

        <div id="emptyState">
            <div class="text-7xl opacity-20 mb-5">🎲</div>
            <div class="text-2xl text-gray-400">尚未建立字庫</div>
            <div class="text-sm text-gray-600 mt-3">
                請先輸入關鍵字並生成日K高頻詞。
            </div>
        </div>

        <div id="cardArea" class="hidden">

            <div id="kanaText" class="text-6xl md:text-7xl font-bold text-white kana-
            glow tracking-widest leading-loose"></div>

            <div id="kanjiText" class="text-2xl md:text-3xl text-cyan-400 mt-4"></div>

            <div id="romajiArea" class="mt-10 hidden">
                <div id="romajiText" class="text-3xl font-bold text-emerald-400
            tracking-widest"></div>
                <div id="meaningText" class="text-lg text-gray-400 mt-4"></div>
            </div>

        </div>

    </div>
</div>
</div>
</div>

<div class="pb-6">
    <button
        id="drawBtn"
        class="px-14 py-5 rounded-full bg-gradient-to-r from-cyan-600 to-blue-600
        hover:from-cyan-500 hover:to-blue-500 text-2xl font-bold tracking-widest shadow-2xl
        active:scale-95 transition-all"
    >
        🎲 抽卡
    </button>
</div>

</main>

<!-- 右側 -->
<aside class="w-full lg:w-96 bg-gray-900 border-l border-gray-800 p-5 overflow-y-auto
shrink-0">

    <div class="flex items-center justify-between mb-5">

```

```

        <h2 class="text-lg font-bold text-emerald-400 tracking-wider">
            五十音対照表
        </h2>
    </div>

    <div id="kanaTable" class="grid grid-cols-5 gap-2"></div>

</aside>

</div>

<script>

const STORAGE_KEY = 'kana_trainer_vocab_v2';
const API_KEY_STORAGE = 'kana_trainer_api_key';

const MODELS = [
    'gemini-2.0-flash-lite',
    'gemini-2.0-flash',
    'gemini-1.5-flash-latest'
];

const kanaData = [
    ['あ', 'a'], ['い', 'i'], ['う', 'u'], ['え', 'e'], ['お', 'o'],
    ['か', 'ka'], ['き', 'ki'], ['く', 'ku'], ['け', 'ke'], ['こ', 'ko'],
    ['さ', 'sa'], ['し', 'shi'], ['す', 'su'], ['せ', 'se'], ['そ', 'so'],
    ['た', 'ta'], ['ち', 'chi'], ['つ', 'tsu'], ['て', 'te'], ['と', 'to'],
    ['な', 'na'], ['に', 'ni'], ['ぬ', 'nu'], ['ね', 'ne'], ['の', 'no'],
    ['は', 'ha'], ['ひ', 'hi'], ['ふ', 'fu'], ['へ', 'he'], ['ほ', 'ho'],
    ['ま', 'ma'], ['み', 'mi'], ['む', 'mu'], ['め', 'me'], ['も', 'mo'],
    ['や', 'ya'], ['ゆ', 'yu'], ['よ', 'yo'],
    ['ら', 'ra'], ['り', 'ri'], ['る', 'ru'], ['れ', 're'], ['ろ', 'ro'],
    ['わ', 'wa'], ['を', 'wo'], ['ん', 'n'],
    ['が', 'ga'], ['ぎ', 'gi'], ['ぐ', 'gu'], ['げ', 'ge'], ['ご', 'go'],
    ['ざ', 'za'], ['じ', 'ji'], ['ず', 'zu'], ['ぜ', 'ze'], ['ぞ', 'zo'],
    ['だ', 'da'], ['ぢ', 'ji'], ['づ', 'zu'], ['で', 'de'], ['ど', 'do'],
    ['ば', 'ba'], ['び', 'bi'], ['ぶ', 'bu'], ['べ', 'be'], ['ぼ', 'bo'],
    ['ぱ', 'pa'], ['ぴ', 'pi'], ['ぷ', 'pu'], ['ぺ', 'pe'], ['ぽ', 'po']
];

const apiKeyInput = document.getElementById('apiKeyInput');
const keywordInput = document.getElementById('keywordInput');
const fetchBtn = document.getElementById('fetchBtn');
const clearBtn = document.getElementById('clearBtn');
const vocabCount = document.getElementById('vocabCount');
const drawBtn = document.getElementById('drawBtn');
const statusBox = document.getElementById('statusBox');
const kanaTable = document.getElementById('kanaTable');
const toggleKeyBtn = document.getElementById('toggleKeyBtn');
const delayRevealToggle = document.getElementById('delayRevealToggle');

```

```

const emptyState = document.getElementById('emptyState');
const cardArea = document.getElementById('cardArea');
const kanaText = document.getElementById('kanaText');
const kanjiText = document.getElementById('kanjiText');
const romajiArea = document.getElementById('romajiArea');
const romajiText = document.getElementById('romajiText');
const meaningText = document.getElementById('meaningText');

let vocabList = [];
let currentCard = null;

function log(...args) {
  console.log('[KanaTrainer]', ...args);
}

function setStatus(text, type='info') {

  statusBox.className = 'rounded-xl px-4 py-3 text-sm border';

  if (type === 'error') {
    statusBox.classList.add('bg-red-950', 'border-red-800', 'text-red-300');
  }
  else if (type === 'success') {
    statusBox.classList.add('bg-emerald-950', 'border-emerald-800', 'text-emerald-300');
  }
  else {
    statusBox.classList.add('bg-gray-900', 'border-gray-800', 'text-gray-300');
  }

  statusBox.innerText = text;
}

function saveVocab() {
  localStorage.setItem(STORAGE_KEY, JSON.stringify(vocabList));
}

function loadVocab() {
  const data = localStorage.getItem(STORAGE_KEY);

  if (!data) return;

  try {
    vocabList = JSON.parse(data);
  } catch(e) {
    log(e);
    vocabList = [];
  }
}

function updateUI() {
  vocabCount.innerText = vocabList.length;
}

```

```

    if (vocabList.length === 0) {
      emptyState.classList.remove('hidden');
      cardArea.classList.add('hidden');
    }
  }

  function renderKanaTable() {

    kanaTable.innerHTML = kanaData.map(item => `
      <div class="bg-gray-950 border border-gray-800 rounded-lg p-2 text-center hover:border-emerald-500 transition">
        <div class="text-xl text-white font-bold">${item[0]}</div>
        <div class="text-xs text-emerald-400 mt-1">${item[1]}</div>
      </div>
    `).join('');
  }

  function randomWeightedCard() {

    if (vocabList.length === 0) return null;

    const pool = [];

    vocabList.forEach(item => {

      let weight = 1;

      if (item.kana.length <= 3) weight += 2;
      if (item.frequency === 'high') weight += 2;

      for (let i=0; i<weight; i++) {
        pool.push(item);
      }
    });

    return pool[Math.floor(Math.random() * pool.length)];
  }

  async function revealAnswer(card) {

    if (delayRevealToggle.checked) {
      await new Promise(r => setTimeout(r, 3000));
    }

    romajiText.innerText = card.romaji || '';
    meaningText.innerText = card.meaning || '';

    romajiArea.classList.remove('hidden');
  }

```



```

async function drawCard() {

  if (vocabList.length === 0) {
    setStatus('請先建立字庫。', 'error');
    return;
  }

  let card;

  do {
    card = randomWeightedCard();
  }
  while (
    vocabList.length > 1 &&
    currentCard &&
    card.kana === currentCard.kana
  );

  currentCard = card;

  emptyState.classList.add('hidden');
  cardArea.classList.remove('hidden');

  kanaText.innerText = card.kana || '';
  kanjiText.innerText = card.kanji || '';

  romajiArea.classList.add('hidden');

  revealAnswer(card);
}

```

```
function buildPrompt(keyword) {
```

```

return `
你是一位日文語感與日K字幕專家。

```

```

目標：
幫助中文使用者將日文五十音訓練成像注音符號般的直覺認讀。

```

```

請根據以下關鍵字：

```

```

${keyword}

```

```

聯想並整理：

```

1. 日K高頻出現
2. JPOP常見
3. 動漫常見
4. 適合五十音視覺辨識
5. 適合快速發音練習

的日文單字或短詞。

請避免：

- 冷門詞
- 過長句子
- 艱深文法

請只輸出 JSON Array。

格式：

```
[
  {
    "kanji": "",
    "kana": "",
    "romaji": "",
    "meaning": ""
  }
]
```

不要輸出 markdown。

不要輸出 `` `json`

不要輸出說明

```
`;
}
```

```
function cleanGeminiText(text) {
```

```
  return text
    .replace(/`` `json/gi, "")
    .replace(/`` `/gi, "")
    .replace(/`` `$/gi, "")
    .trim();
}
```

```
async function fetchWithRetry(url, options, retries=3) {
```

```
  for (let i=0;i<retries;i++) {
```

```
    const response = await fetch(url, options);
```

```
    if (response.status !== 429) {
      return response;
    }
```

```
    const delay = 1000 * Math.pow(2, i);
```

```
    setStatus(` Gemini API 過載， ${delay/1000} 秒後重試... `);
```

```
    await new Promise(r => setTimeout(r, delay));
```

```
  }
```

```

    throw new Error('Gemini API 過載中');
  }

  async function fetchWords() {

    const apiKey = apiKeyInput.value.trim();
    const keyword = keywordInput.value.trim();

    if (!apiKey) {
      setStatus('請輸入 Gemini API Key', 'error');
      return;
    }

    if (!keyword) {
      setStatus('請輸入關鍵字', 'error');
      return;
    }

    localStorage.setItem(API_KEY_STORAGE, apiKey);

    fetchBtn.disabled = true;

    try {

      let finalResponse = null;
      let lastError = null;

      for (const model of MODELS) {

        try {

          setStatus(`正在使用 ${model} 建立字庫...`);

          const apiUrl = `https://generativelanguage.googleapis.com/v1beta/models/${model}:generateContent?key=${apiKey}`;

          log('API URL', apiUrl);

          const response = await fetchWithRetry(apiUrl, {
            method: 'POST',
            headers: {
              'Content-Type': 'application/json'
            },
            body: JSON.stringify({
              contents: [
                {
                  parts: [
                    {
                      text: buildPrompt(keyword)
                    }
                  ]
                }
              ]
            })
          });

```

```

        ]
      }
    ]
  })
});

const data = await response.json();

log(data);

if (!response.ok) {
  throw new Error(data.error?.message || 'Gemini API Error');
}

if (!data.candidates?.length) {
  throw new Error('Gemini 無有效回應');
}

finalResponse = data;

break;

} catch (err) {
  log(err);
  lastError = err;
}
}

if (!finalResponse) {
  throw lastError || new Error('所有模型皆失敗');
}

let rawText = finalResponse.candidates[0].content.parts[0].text;

rawText = cleanGeminiText(rawText);

log(rawText);

const arr = JSON.parse(rawText);

if (!Array.isArray(arr)) {
  throw new Error('JSON 格式錯誤');
}

const existing = new Set(
  vocabList.map(v => `${v.kana}_${v.kanji}`)
);

let inserted = 0;

arr.forEach(item => {

```

```

const uniqueKey = `${item.kana}_${item.kanji}`;

if (!existing.has(uniqueKey)) {

  vocabList.push({
    ...item,
    createdAt: Date.now(),
    sourceKeyword: keyword,
    frequency: 'high'
  });

  inserted++;
}
});

saveVocab();
updateUI();

setStatus(`成功新增 ${inserted} 個新單字。`, 'success');

keywordInput.value = '';

} catch (err) {

  console.error(err);

  setStatus(`失敗：${err.message}`, 'error');

} finally {
  fetchBtn.disabled = false;
}
}

function clearVocab() {

  const ok = confirm('確定清空字庫? ');

  if (!ok) return;

  vocabList = [];

  localStorage.removeItem(STORAGE_KEY);

  updateUI();

  setStatus('字庫已清空');
}

function init() {

```

```

renderKanaTable();

loadVocab();

updateUI();

const savedKey = localStorage.getItem(API_KEY_STORAGE);

if (savedKey) {
  apiKeyInput.value = savedKey;
}

setStatus('系統初始化完成');
}

fetchBtn.addEventListener('click', fetchWords);
clearBtn.addEventListener('click', clearVocab);
drawBtn.addEventListener('click', drawCard);

toggleKeyBtn.addEventListener('click', () => {

  if (apiKeyInput.type === 'password') {
    apiKeyInput.type = 'text';
    toggleKeyBtn.innerText = '🔒';
  }
  else {
    apiKeyInput.type = 'password';
    toggleKeyBtn.innerText = '👁️';
  }
});

init();

</script>

</body>
</html>

```